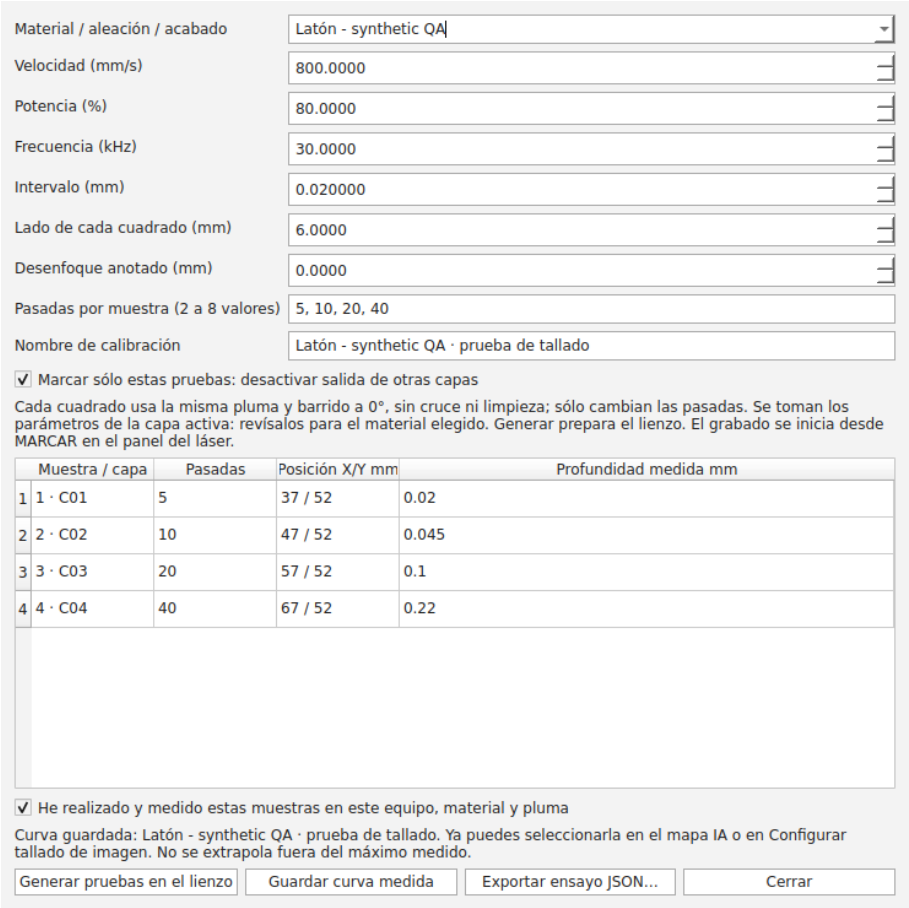
Pruebas de tallado por material

Abre Laser - Pruebas de tallado por material. Elige material y acabado, revisa la pluma de la capa activa e introduce entre 2 y 8 cantidades de pasadas, como 5, 10, 20 y 40. Seleccionar material no aplica una receta automáticamente. Generar prepara cuadrados centrados en el campo, cada uno en su capa, con idéntica pluma y barrido a 0 grados, sin cruce ni limpieza.



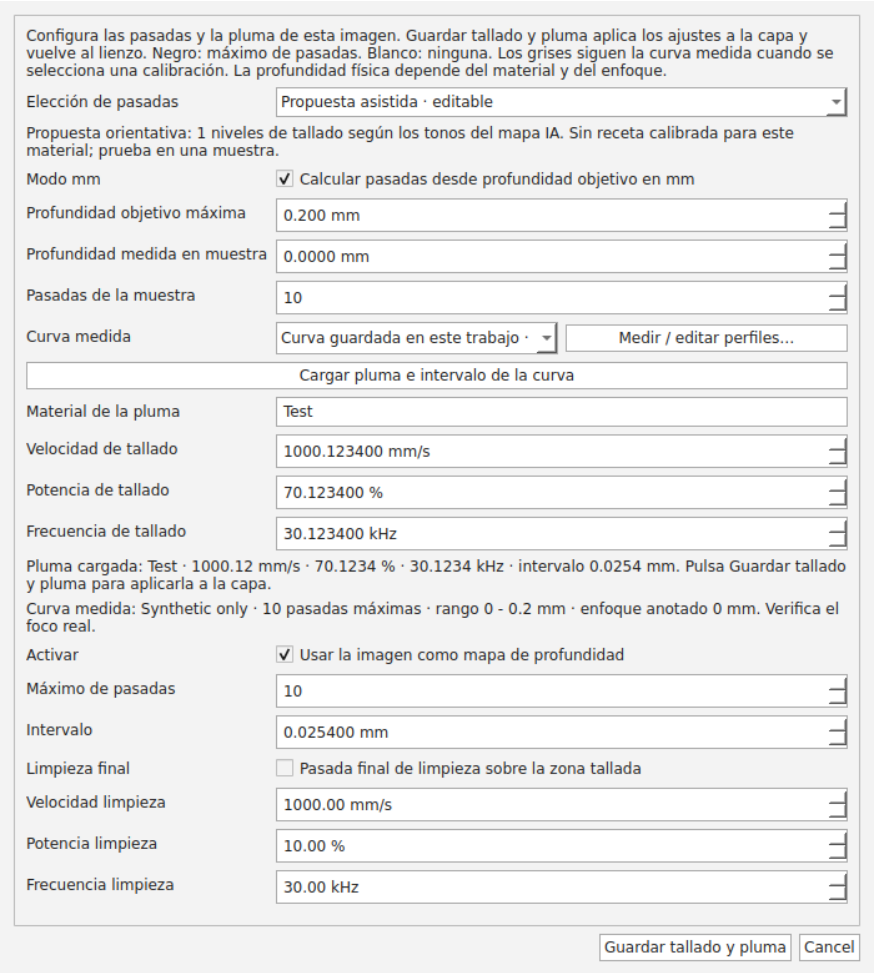
La tabla identifica muestras y posiciones; los textos no se graban. Marcar sólo estas pruebas desactiva la salida de otras capas conservando sus objetos. Cierra la ventana, coloca una muestra del material, revisa ENCUADRAR y pulsa MARCAR. Se procesan de seguido con el contador y ETA existentes.

Vuelve a esta ventana durante la misma sesión, introduce profundidades medidas, confirma su origen y guarda la curva. Faltan mediciones, receta cambiada o geometría modificada bloquean guardar. Los ensayos JSON se conservan en work/DEPTH_TESTS. Guarda también el proyecto. La tabla no se recarga automáticamente tras reiniciar; las curvas guardadas sí se recuperan.

Captura con mediciones sintéticas usadas para probar el programa; no son resultados de material. La nueva opción sólo prepara trabajos. Los ensayos físicos y sus mediciones corresponden al usuario. Puedes reutilizar la curva guardada en mapas IA y tallado de imagen dentro de su rango medido.

Enviar al tallado y guardar la pluma

Enviar a tallado termina el generador y abre Preparar tallado de la capa. El generador queda oculto durante la revisión. Cargar pluma e intervalo de la curva muestra los valores en campos editables: material, velocidad, potencia, frecuencia e intervalo. Selecciona primero una curva medida; en modo lineal introduce los valores de tu muestra.



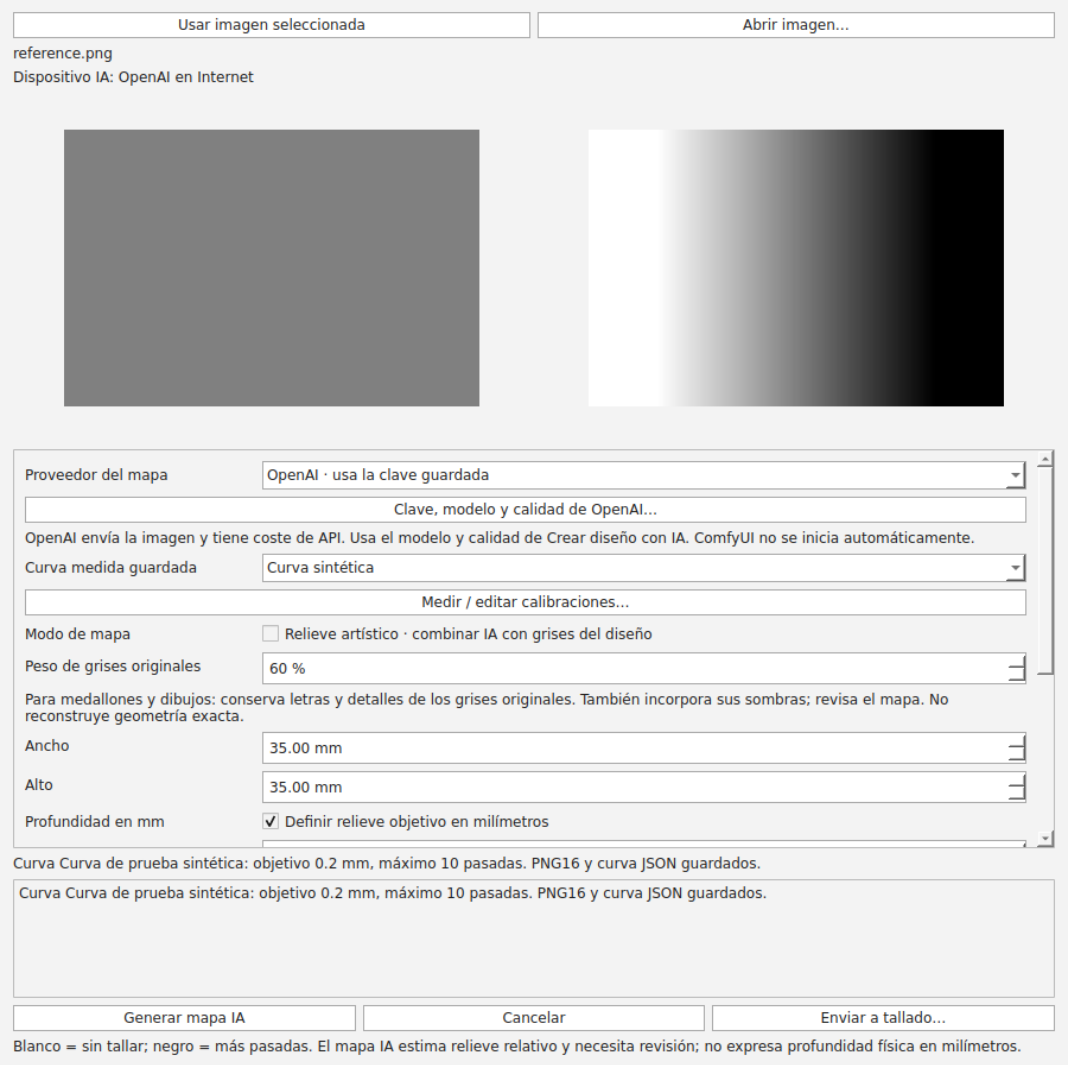
Guardar tallado y pluma está siempre visible debajo del panel desplazable. Aplica imagen, pluma e intervalo juntos y vuelve al lienzo. Cancelar conserva las capas originales. Si falla la validación o guardar el mapa, la ventana sigue abierta sin cambios parciales. Configurar tallado de imagen, en el panel del láser, edita la imagen seleccionada y no genera otro mapa.

La captura utiliza una curva sintética para probar software. Se verificó el envío, carga de pluma, intervalo decimal, guardado en capa, cancelación y fallo de exportación con Qt real y SDK simulado. Sin prueba física del láser.

Clave guardada y mapa calibrado con OpenAI

OpenAI es el proveedor inicial. ComfyUI queda opcional y no arranca ni descarga modelos automáticamente. En Crear diseño con IA introduce tu clave: al salir del campo o generar se guarda cifrada con Windows DPAPI para tu usuario. Se recupera al abrir el programa. Usa Olvidar clave guardada para eliminarla. No viaja en ZIP, proyectos, argumentos ni registros.

Abre Crear mapa de profundidad IA, selecciona una imagen y elige OpenAI. Clave, modelo y calidad se comparten con Crear diseño con IA. Generar envía la referencia a la API de imágenes y produce una estimación artística de relieve relativo; no mide milímetros. Revisa el resultado.



Captura con mapa y curva sintéticos para verificar software. No es un ensayo físico ni un resultado real de OpenAI.

Guardar y usar la curva física

Selecciona Curva medida guardada o usa Medir / editar calibraciones para registrar al menos dos muestras medidas de tu material, pluma, intervalo y enfoque anotado. Introduce el objetivo máximo dentro del rango medido. La inversa por tramos convierte cada tono a pasadas y bloquea objetivos fuera del rango. Las mediciones externas siguen siendo referencias que necesitan ensayos propios.

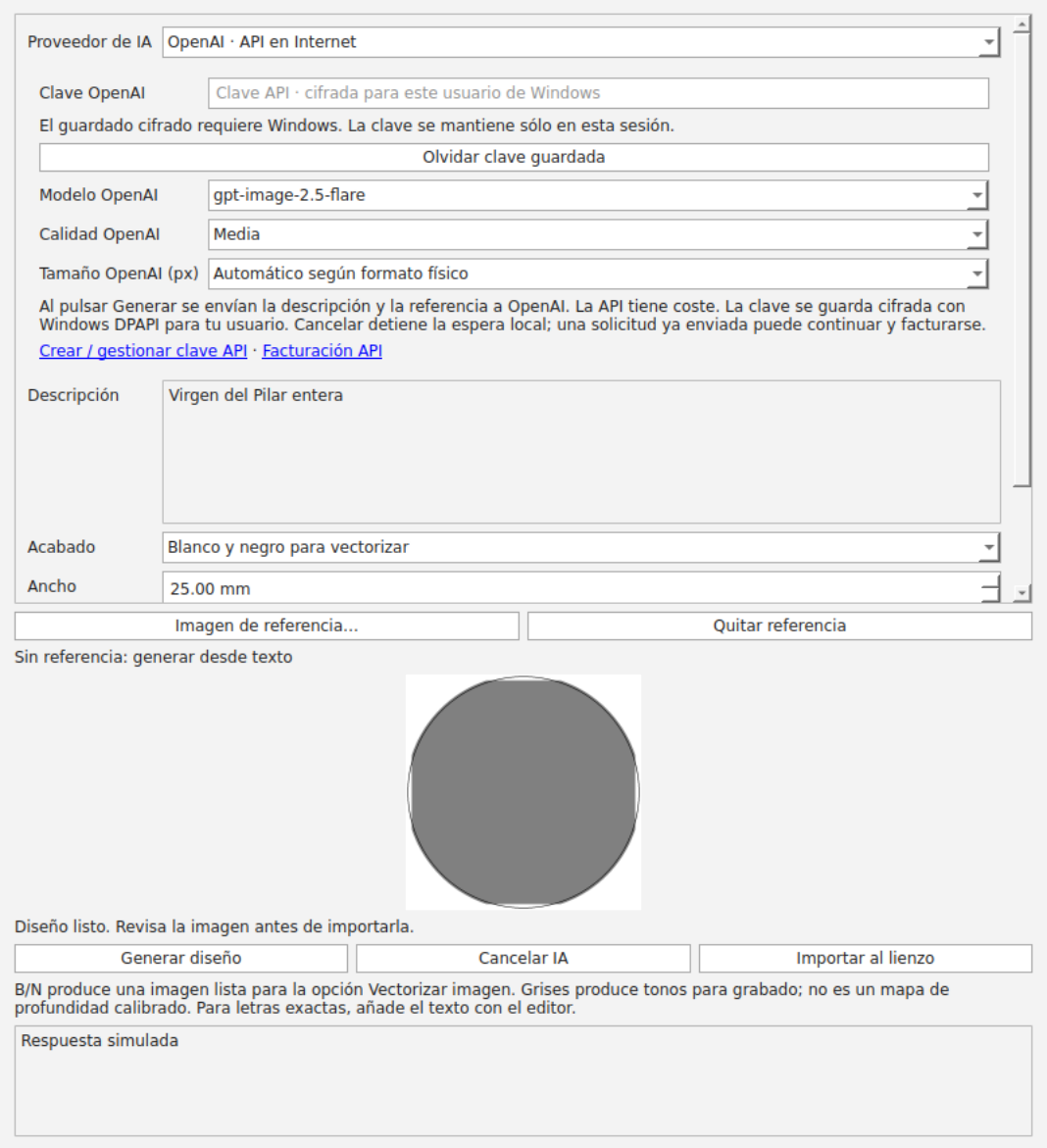
En work/DEPTH_MAPS se conservan el mapa relativo de OpenAI y su JSON, el mapa procesado, el objetivo PNG16, las pasadas PNG16 y el JSON de la curva usada. Puedes reutilizarlos como imágenes. Enviar a tallado conserva la curva en el trabajo: aplica su pluma e intervalo y comprueba material y enfoque. Los cambios se revisan antes de aplicarlos; no hay envío de láser desde el generador.

El cifrado está ligado al usuario de Windows. Si no puede cifrar o descifrar se informa en el campo y se mantiene sólo la clave de sesión; no se escribe una copia en claro. En otro equipo o usuario vuelve a introducir tu clave. La API tiene facturación propia. Cancelar detiene la espera local y no garantiza que se anule un trabajo ya aceptado por el servicio.

Verificación: transporte y proceso Qt simulados, conservación de los tonos de profundidad, uso compartido de clave/modelo, persistencia con cifrador inyectado para probar entradas/salidas, fallo sin sobrescritura, eliminación de clave, PNG16 con curva y bloqueo fuera de rango, ausencia de inicio automático de ComfyUI. No se probaron DPAPI en Windows, la API facturada ni el resultado físico del láser.

IA local u OpenAI desde la misma ventana

Abre Crear diseño con IA y elige OpenAI en Proveedor de IA. Introduce tu clave API, selecciona modelo, calidad y tamaño, y describe el diseño en español. Puedes añadir una imagen de referencia. Generar envía el trabajo; revisa el resultado e Importar al lienzo. Local mantiene Klein y tus modelos descargables.



Captura de la interfaz real con respuesta de prueba simulada: no es un diseño generado realmente por OpenAI. La clave está vacía en esta captura. Los controles de formato físico están en un panel desplazable.

Clave, cuenta y operación

Crea o gestiona la clave desde el enlace de la ventana. Se guarda cifrada con Windows DPAPI para tu usuario al salir del campo o generar; se recupera al abrir el programa. Olvidar clave guardada elimina el archivo y vacía el campo. La clave queda en %LOCALAPPDATA%/43AYE3/openai_key.dpapi, fuera de proyectos y actualizaciones; no se exporta. OPENAI_API_KEY sigue disponible como alternativa. También se recuerdan proveedor, modelo, calidad y tamaño. La API tiene facturación aparte de la suscripción ChatGPT. El acceso al modelo depende de tu proyecto y puede requerir verificación de organización.

Generar envía la descripción y la referencia elegida a OpenAI por HTTPS. Sin referencia usa Images Generations; con referencia usa Images Edits. Calidad baja/media/alta y formato cuadrado/horizontal/vertical se seleccionan en la ventana. Automático elige formato según las dimensiones físicas, con cuadrado para medallones. El cliente recibe PNG y conserva el procesamiento local B/N o grises, borde circular e importación a escala.

Cancelar detiene la espera y el proceso local. Una solicitud que llegó al servicio puede continuar y facturarse. No se reintenta automáticamente tras fallo de red. Comprueba uso/facturación antes de repetir. Errores 401, 403 y 429 se muestran para revisar clave, permisos o cuota. El progreso es indeterminado; no se inventa porcentaje.

Implementación y verificación

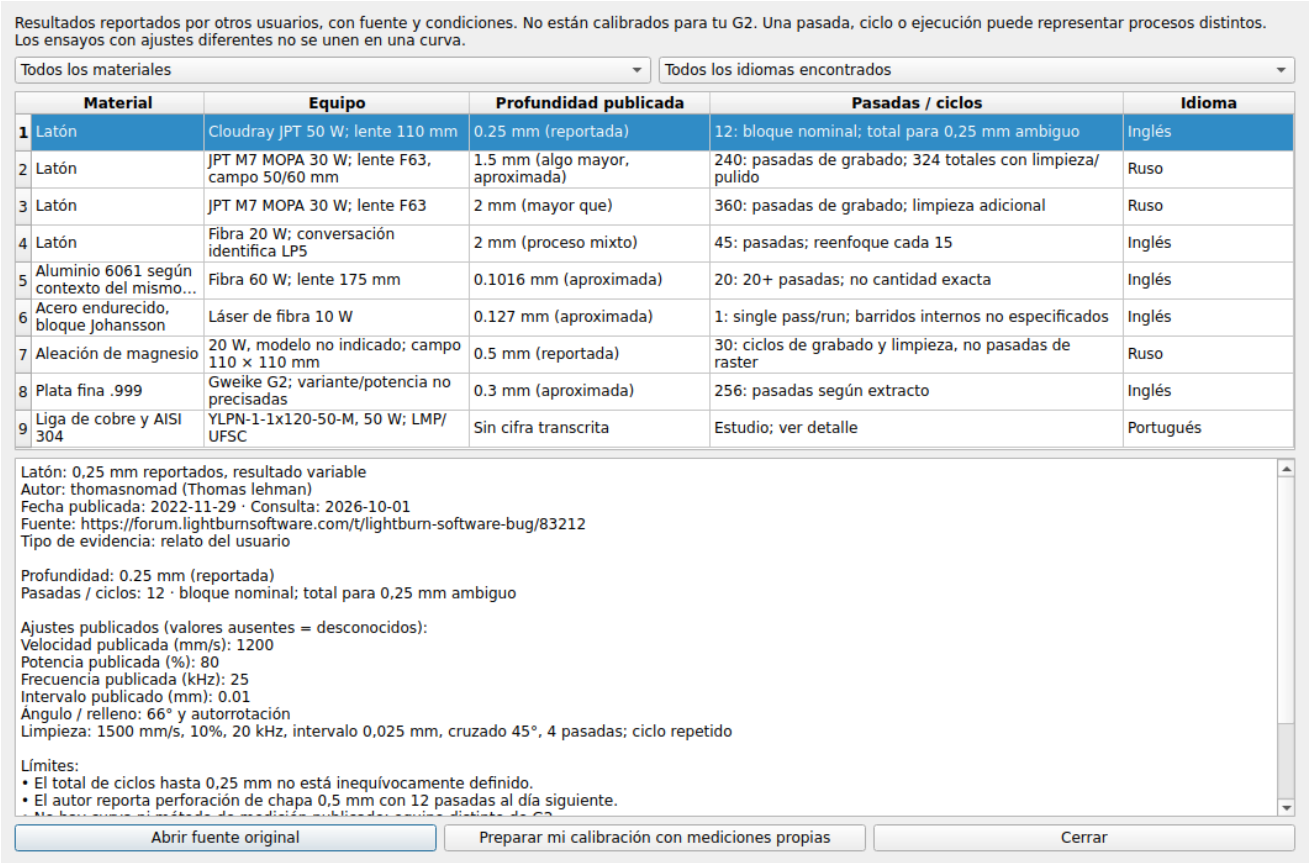
openai_image_client.py prepara JSON o multipart y usa HTTPS al host fijo api.openai.com. openai_image_cli.py recibe los datos por stdin; openai_image_worker.py gestiona un proceso Qt cancelable. No se requiere instalar el SDK OpenAI, ComfyUI ni traductor local para este proveedor. ComfyUI es opcional y se prepara sólo al elegir Local y pulsar Conectar / instalar. La clave se excluye de preferencias, salida JSON, argumentos y registros. El resultado guarda proveedor, prompt, modelo, calidad, tamaño y uso devuelto.

Se probaron respuestas simuladas para texto, referencia y error 401, exclusión de clave, retorno entre proveedores, cancelación y dimensiones importadas. No se realizaron solicitudes reales ni facturadas: falta comprobación con la clave y cuenta del usuario. No hay cambios en envío de órdenes físicas.

Documentación oficial de OpenAI consultada el 01/10/2026:
https://developers.openai.com/api/docs/guides/image-generation
https://learn.chatgpt.com/docs/pricing

Referencias externas de profundidad

Nueva ventana: Tallado imagen - Gestionar calibraciones - Mediciones publicadas de otros usuarios. Incluye ocho resultados declarados por usuarios y un estudio experimental. Se buscó en nueve idiomas: inglés, español, alemán, francés, ruso, chino, japonés, italiano y portugués. No es un rastreo exhaustivo de todos los idiomas o comunidades privadas.



Filtra por material e idioma y selecciona una ficha. Se muestran equipo, profundidad, contador publicado, ajustes conocidos, autor, fuente y límites. Abrir fuente original abre el navegador. El informe REFERENCIAS_PROFUNDIDAD_WEB.md contiene los enlaces del catálogo.

La cifra publicada se conserva como declarada, aproximada, límite o proceso mixto. El caso G2/plata procede sólo de un extracto del buscador; la página completa no pudo recuperarse. El estudio portugués tiene medición instrumental, pero no se transcriben números de gráficos que no se pudieron inspeccionar.

Crear una calibración propia desde una referencia

Preparar mi calibración sólo copia nombre y material, guarda el enlace de origen y conserva la pluma que tenías abierta. Los puntos quedan vacíos. Añade al menos dos mediciones obtenidas con tu equipo, material y pluma; confirma Mis puntos son mediciones realizadas en este equipo antes de guardar.

No se ha encontrado una curva completa y reproducible del G2 Pro 30 W en las fuentes recuperadas. La aplicación no convierte estas referencias en calibraciones locales ni mezcla ensayos con cambios de foco, potencia, lente, limpieza o estrategia. Cada ciclo o ejecución puede incluir varios barridos: no se convierte sin más en mm/pasada.

Los perfiles guardan reference_origin con identificador, URL, consulta y confirmación de medición local. depth_calibration.py bloquea el uso de un borrador externo sin esa confirmación. external_depth_references.json contiene el catálogo separado de depth_calibrations.json, que guarda tus perfiles. Los mapas exportados mantienen la referencia dentro de la calibración.

Pruebas: esquema de fuentes, filtros Qt, catálogo sin curvas utilizables, borrador sin puntos, bloqueo previo a medición propia, persistencia del origen y reinicio limpio al crear un perfil nuevo. Se vuelven a comprobar mapas, trayectorias, modelos y panel de marcado. Sin nuevas pruebas físicas del láser.

43AYE3

Modelos y mapas calibrados

FLUX.2 Klein 4B FP8 · Español · Profundidad en mm · Contador y ETA

Revisión vigente del 1 de octubre de 2026. Base recuperada: 43AYE3_COMPLETO_SDXL_ESPANOL_CONTADOR.zip. Este documento incorpora el cambio posterior a FLUX.2 Klein para una GPU de 8 GB.

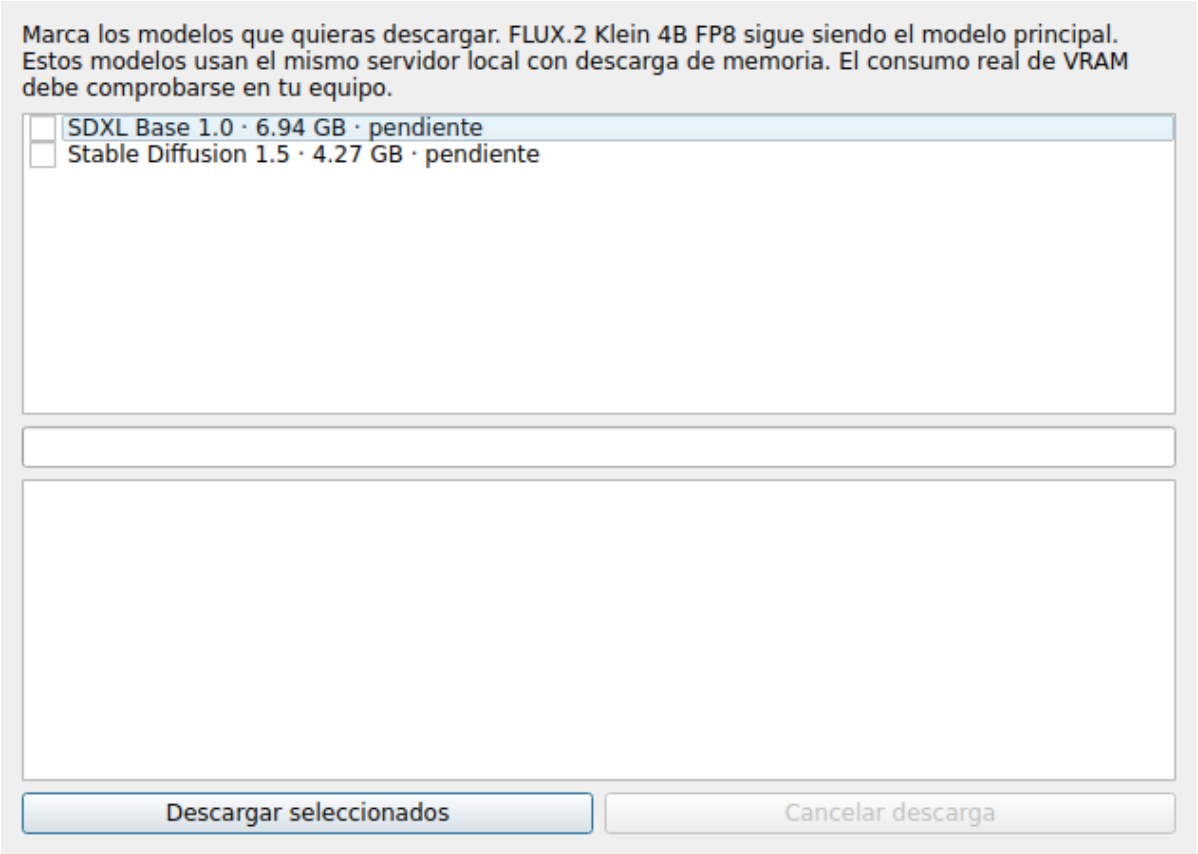
Cómo leer este PDF: la primera parte describe la entrega actual. Después se conserva la referencia completa del editor y API de la revisión anterior. Para modelo IA, envío de marcado, ETA y punto inicial de latón, prevalece esta primera parte.

Las capturas nuevas son de la aplicación real en Qt, ejecutada sin hardware y con confirmaciones SDK simuladas. No muestran una prueba de grabado físico ni una generación real de Klein.

Estado físico pendiente: el fallo comunicado «NO MARCA» no tiene todavía causa confirmada. La sesión única experimental sigue desactivada; se usa envío compatible por bloque. No se afirma que el tallado físico ya sea fluido.

Ventana independiente con casillas

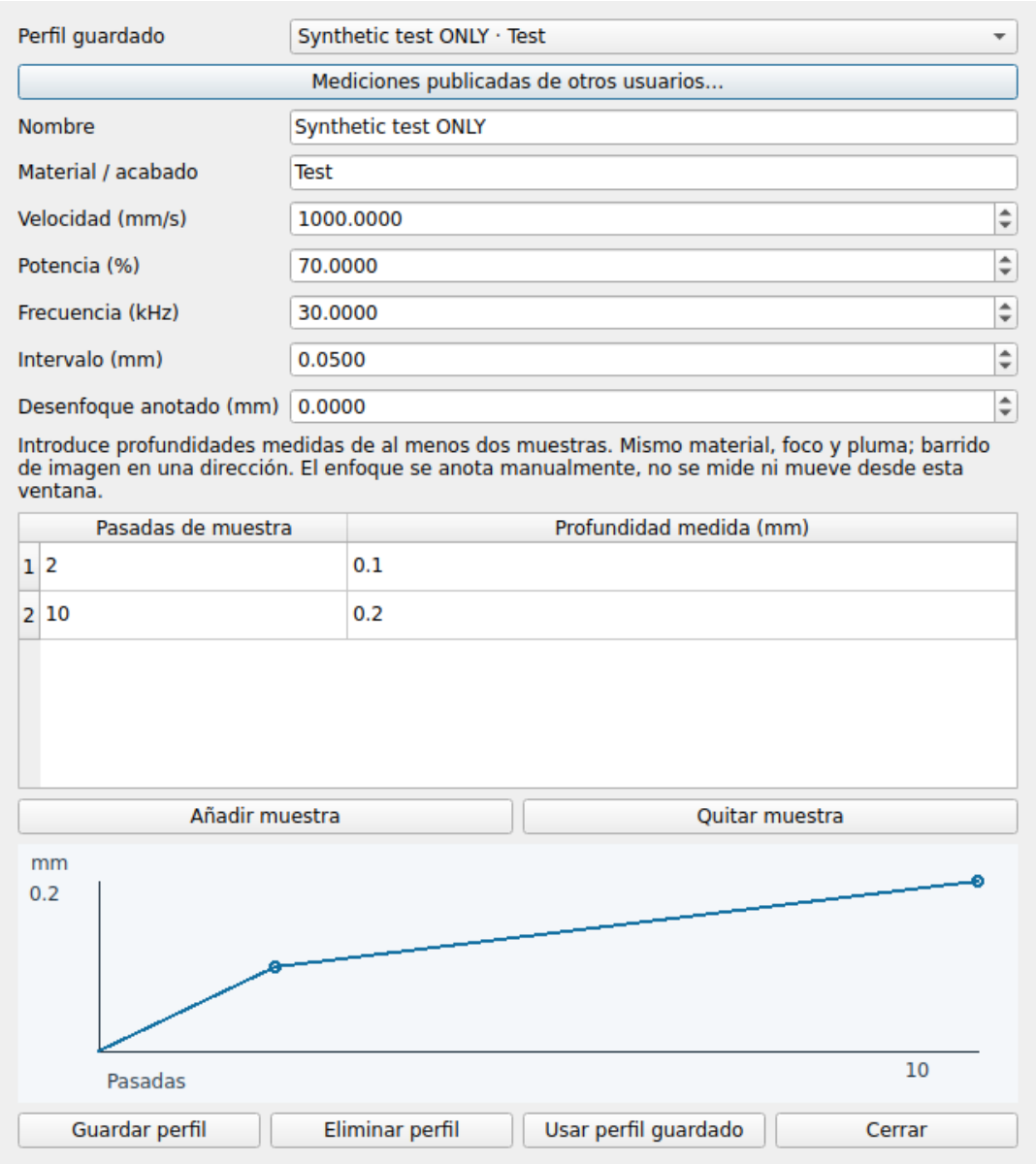
En Crear diseño con IA pulsa Descargar otros modelos. Marca SDXL Base 1.0 o Stable Diffusion 1.5 y pulsa Descargar seleccionados. La ventana muestra tamaño, progreso y cancelación. Se comprueba SHA256 antes de publicar cada archivo. Klein permanece como modelo principal. Un archivo parcial se reinicia al reintentar; un modelo ya presente se verifica antes de reutilizarlo.



Las descargas requieren conexión y espacio en disco. El perfil del servidor usa lowvram y VAE en CPU. No se ha probado el consumo real de estos pesos en la RTX de 8 GB. Si falla CUDA, reduce resolución o usa Klein a 512 px.

Calibración con una curva medida

Abre Tallado imagen y Gestionar calibraciones. Introduce nombre, material, velocidad, potencia, frecuencia, intervalo y enfoque anotado. Añade al menos dos muestras con pasadas y profundidad medida; guarda y selecciona el perfil. El enfoque requiere ajuste y comprobación física, no se cambia automáticamente.



La captura muestra datos sintéticos de prueba de software. No es una receta de material ni se distribuye como perfil medido.

Aplicar y comprobar el relieve

1. Selecciona la curva guardada y activa profundidad en mm.
2. Aplica la pluma y el intervalo de esa calibración.
3. Introduce el objetivo dentro del máximo medido.
4. Revisa el mapa y confirma Tallado imagen.
5. Antes de marcar, verifica material y enfoque; mide el resultado.

La IA aporta relieve relativo. Cada tono se convierte en profundidad objetivo y después en pasadas mediante la inversa de la curva por tramos. Las pasadas fraccionarias se redondean hacia arriba; el resultado está cuantizado por pasadas y necesita verificación. Blanco no recibe pasadas. No se extrapola más allá del máximo medido. La opción lineal de una muestra se conserva como estimación antigua.

Se exportan PNG16 de objetivo y de pasadas, junto con JSON que guarda la curva. El generador de trayectorias usa esa misma curva y bloquea discrepancias de material o pluma. La limpieza se ejecuta aparte y puede modificar el resultado final; incluye su efecto al verificar tu proceso.

Módulos y pruebas

model_downloads.py contiene URLs oficiales con revisión y SHA256 fijados. model_manager_ui.py ejecuta las descargas en un proceso cancelable. depth_calibration.py valida, guarda, invierte y exporta curvas. depth_calibration_ui.py edita perfiles y muestra su gráfica. No se envían órdenes de láser desde estas ventanas.

Pruebas: inversión no lineal, rango, PNG16/pasadas, persistencia, descarga atómica con contenido simulado y SHA256 incorrecto, casillas Qt y cancelación sin cambios de pluma. Se conserva la prueba del panel con SDK simulado. No se han ejecutado los pesos reales ni el láser físico.

43AYE3

Programación y API actualizadas

FLUX.2 Klein 4B FP8 · Español · Profundidad en mm · Contador y ETA

Revisión vigente del 1 de octubre de 2026. Base recuperada: 43AYE3_COMPLETO_SDXL_ESPANOL_CONTADOR.zip. Este documento incorpora el cambio posterior a FLUX.2 Klein para una GPU de 8 GB.

Cómo leer este PDF: la primera parte describe la entrega actual. Después se conserva la referencia completa del editor y API de la revisión anterior. Para modelo IA, envío de marcado, ETA y punto inicial de latón, prevalece esta primera parte.

Las capturas nuevas son de la aplicación real en Qt, ejecutada sin hardware y con confirmaciones SDK simuladas. No muestran una prueba de grabado físico ni una generación real de Klein.

Estado físico pendiente: el fallo comunicado «NO MARCA» no tiene todavía causa confirmada. La sesión única experimental sigue desactivada; se usa envío compatible por bloque. No se afirma que el tallado físico ya sea fluido.

Arquitectura del generador Klein

Módulo	Responsabilidad
flux_klein.py	Archivos y grafo nativo FLUX.2 4B FP8, texto y edición por referencia
comfy_setup.py	Actualiza ComfyUI gestionado, prepara 3 archivos con revisión/hash oficial, traductor y CUDA
comfy_design_client.py	Descubre componentes, genera grafo y gestiona cola y salida
comfy_design_ui.py	Prompt español, resolución 768/512, edición e importación
mark_time_estimate.py	ETA por longitudes, bloques terminados y STOP
mark_pass_counter.py	Pasadas completadas por segmento, incluida limpieza

Nodos de Klein: UNETLoader FP8, CLIPLoader tipo flux2 con device cpu, VAELoader, EmptyFlux2LatentImage, Flux2Scheduler de 4 pasos, CFGGuider cfg=1, Euler y SamplerCustomAdvanced. La referencia usa ReferenceLatent; la decodificación usa VAEDecodeTiled. El servidor utiliza cpu-vae y lowvram.

Estas conexiones se contrastaron con el workflow destilado oficial de Comfy-Org. Son un grafo API plano, sin dependencia de nodos personalizados ni un CheckpointLoaderSimple para Klein.

Ejemplo offline: generar workflow API

```
from comfy_design_client import workflow, DEFAULT_MODEL
graph = workflow(
    'Virgin on her column', 'Escala de grises',
    DEFAULT_MODEL, 768, 768, steps=4,
    circular=True, seed=43)
# No se inicia ni se dispara el laser.
```

Ejecuta DOCUMENTACION/ejemplos/demo_flux_klein.py. Crea demo_flux_klein_api.json sin descargar modelos ni iniciar ComfyUI. Para ejecutar un grafo, la aplicación usa su servidor local ya preparado; el ejemplo no lo inicia.

Ejemplo offline: mm y pasadas

```
from depth_map_tools import passes_from_mm, export_mm_map
passes, rate = passes_from_mm(.2, .1, 10)
assert passes == 20
path = export_mm_map('grises.png', .2, .1, 10)
# PNG uint16 + JSON con target_max_mm.
# profundidad_mm = pixel / 65535 * target_max_mm
```

El JSON declara desired_depth_not_measurement. Los valores positivos y finitos se validan antes de escribir el mapa. El historial de pluma se conserva para detectar cambios al marcar.

Ejemplo offline: ETA y STOP

```
from mark_time_estimate import MarkTimeEstimate
from mark_pass_counter import PassCounter
groups = [(0, [[(0,0),(100,0)]], 2, 10., 25., 30.)]
eta = MarkTimeEstimate(groups)
count = PassCounter([(1, 2, 'grabado')])
# Solo tras retorno correcto del bloque:
eta.confirm_block(); count.confirm_block()
print(count.text(), eta.text())
# Ante parada antes de completar: eta.stop()
```

El porcentaje pesa las longitudes de bloques terminados. No avanza por reloj. La ETA usa un reloj monotónico para medir duración y hora local para el final aproximado; STOP congela duración y no confirma bloques parciales.

Ejecuta demo_contador_eta.py para simular grabado seguido de limpieza y parada parcial. Es independiente del SDK y no emite órdenes de máquina.

Verificación y límites reales

Se comprobaron sintaxis, grafo Klein con y sin referencia, CPU del codificador, cálculo mm/PNG16, validación de entradas, fondo conectado, contador y ETA. La interfaz Qt real se probó con un proceso nativo simulado: fin correcto y cancelación previa al envío.

No se ejecutaron pesos Klein/OPUS-MT aquí, ni se compiló MSVC ni se probó USB/G2. Las fuentes SDK se conservan; el comando activo sigue siendo --mark-vector-design por bloque. La sesión única descrita en la referencia histórica siguiente no está activa.

Fuentes: https://docs.comfy.org/tutorials/flux/flux-2-klein
https://huggingface.co/black-forest-labs/FLUX.2-klein-4b-fp8
https://github.com/Comfy-Org/workflow_templates

43AYE3

Guía de programación y API

Arquitectura, modelos de datos, ejemplos y referencia del código completo del proyecto.

Edición: G2 V1 - Relieve 5 - GPU / CPU - profundidad objetivo en mm - Help / Ayuda.

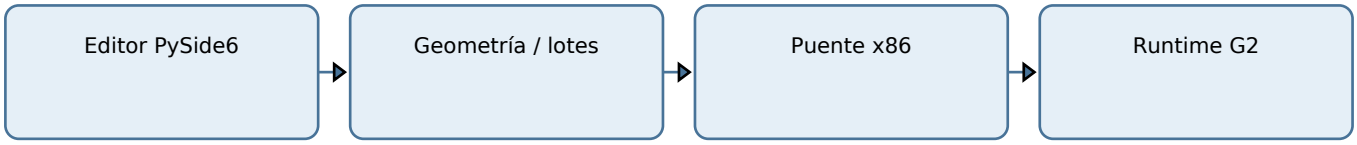
Base documental: ZIP 43AYE3_LASER_PYTHON_INCLUIDO_REDONDO (1).zip y cambios de esta entrega. Revisión del 1 de octubre de 2026. Incluye el código actual de Relieve 5, ayuda integrada y cola de marcado con sesión única.

Las capturas proceden de la interfaz real ejecutada sin conexión física a la máquina y de capturas proporcionadas por el usuario, identificadas en sus pies. Los dibujos y mapas de ejemplo son demostraciones; no son resultados de grabado ni mediciones reales.

Contenido

1. Arquitectura y entornos
2. Objetos, capas y persistencia
3. Construcción de trabajos
4. Relleno, raster y profundidad
5. IA y selección de dispositivo
6. Mapa objetivo en mm
7. Interfaz y nuevas herramientas
8. Puente nativo y API C
9. Compilación y diagnóstico
10. Ejemplos ejecutables y validación
11. Referencia de módulos y archivos

1. Arquitectura y entornos



laser_studio.Main compone el editor PySide6 y llama a instaladores de interfaz. El panel de producción está en production_laser_panel.py. Los cálculos de imagen se despachan a tareas; ComfyUI se gestiona por un servidor HTTP local supervisado.

El editor y la IA utilizan Python de 64 bits. La integración del fabricante trabaja con puentes x86: no cargues una DLL de 32 bits dentro de Python de 64 bits. El programa llama a ejecutables auxiliares y usa archivos para preparar lotes.

G2_RUNTIME contiene DLL y datos requeridos por la versión nativa. BSL_SDK/43AYE3_G2_SDK contiene la capa de compatibilidad y el selftest. native_bridge_x86 contiene clientes C++ de diagnóstico y preparación.

Las funciones de dispositivo tienen efectos físicos; los exemplos Python de esta guía construyen geometría y archivos, y no llaman a MARCAR. El código de interfaz puede iniciar preparación IA al entrar en el bucle de eventos: desactívala explícitamente en herramientas de documentación offline.

2. Objetos, capas y persistencia

Cut almacena parámetros de pluma. GItem contiene QPainterPath y metadatos; RasterItem contiene el original, la vista previa y image_settings. GroupItem reúne hijos y conserva transformaciones. La escena es QGraphicsScene.

Main._save produce JSON versión 3 con field, cuts, groups e items. La escritura utiliza un temporal, flush y fsync antes de os.replace. Los raster se referencian por source; el proyecto no incrusta sus bytes.

Los parámetros de imagen y de mm se guardan con el objeto, no globalmente. El material se conserva en la pluma. Generar una matriz amplía cuts mediante ensure_cut_capacity; nunca asumas que hay solo 16 capas.

Para asignar una capa actualiza item.layer, la fila y la vista. set_layer(..., assign_selection=False) cambia la capa activa sin reasignar accidentalmente los objetos seleccionados.

Objeto / campo	Contrato
Cut.dict()	Copia del diccionario de atributos de la pluma.
GItem.data43()	Tipo, capa, transformaciones, puntos y metadatos.
RasterItem.data43()	Ruta fuente, capa, transformación e image_settings.
GroupItem.data43()	ID, padre y transformaciones del grupo.
Main.selected()	Objetos seleccionados para acciones del editor.

Ejemplo: construir una pluma y una figura

```
from PySide6.QtCore import QRectF
from PySide6.QtGui import QPainterPath
from laser_studio import Cut, GItem

cut = Cut(0)
cut.mode = 'Fill'
cut.speed, cut.power, cut.frequency = 1000, 20, 30
cut.interval, cut.passes = 0.05, 1
path = QPainterPath()
path.addRect(QRectF(45, 45, 20, 20))
item = GItem(path)
item.layer = 0
main.scene.addItem(item)
main.cuts[0] = cut
main.update_cut_row(0)
main.changed = True
main.schedule_preview()
```

Los valores son un ejemplo de configuración del editor, no una receta material. Se necesita una QApplication antes de crear objetos gráficos. El ejemplo completo offline se incluye en DOCUMENTACION/ejemplos/demo_trabajo.py.

3. Construcción de trabajos

laser_job.from_scene(main, selected_only=False, use_cache=False) produce LaserJob: operations, bounds y warnings. JobOperation contiene kind, layer, geometry, speed, power, frequency, passes e interval.

Las imágenes y mapas alternan sentido en filas consecutivas conservando todos los segmentos, para reducir retornos sin emisión. La vista previa usa estas operaciones para representar Line, Fill e Image. El marcado real prepara los lotes en production_laser_panel.py con g2_vector_frame.py; no debes asumir que llamar from_scene ejecuta la máquina.

layer_mark_paths prepara recorridos vectoriales; raster_mark_paths prepara tramos de píxeles oscuros; depth_map_paths produce recorridos por nivel de profundidad. backend_batches adapta la geometría a la capacidad del puente.

El panel ordena capas por priority y prepara grupos de marcado; los grupos de limpieza se agregan después de los de tallado. mark_queue.write_mark_queue conserva su orden y plumas. --mark-vector-queue valida todos los bloques antes de conectar, reutiliza conexión y marcador, y mantiene un evento STOP durante toda la cola. El log se escribe a archivo para evitar bloquear una tubería de salida. Un resultado nativo correcto permite pasar al siguiente lote; fallo o STOP cancela lo restante.

Función	Uso
from_scene(main, selected_only=False, use_cache=False)	Preparar trabajo para revisión.
all_vector_paths(main, layer_filter=None)	Recuperar trayectorias de vectores transformados.
layer_mark_paths(main, layer)	Preparar Line / Fill de una pluma.
backend_batches(paths)	Dividir para el backend.
depth_map_paths(item, passes, interval)	Generador de (nivel, trayectorias) para mapas.

4. Relleno, raster y profundidad

hatch_engine.horizontal_hatch(path, interval, angle) obtiene rayado horizontal rotado de un QPainterPath. El Fill cruzado calcula una segunda dirección a angle + 90. Offset Fill sustrae el trazo del borde para producir una región interior.

laser_image_pipeline.prepare devuelve la ruta de una imagen procesada y utiliza caché de argumentos. process_array procesa brillo, oscuridad, contraste, gamma, sharpen, inversión y modo. image_async evita bloquear la interfaz durante cálculos de raster.

depth_map_paths reescala el raster según tamaño físico / intervalo. En el nivel k conserva píxeles cuyo gris sea menor que 255 - (k-1)*255/N. Equivale a techo((255-gris)*N/255) pasadas por píxel; blanco recibe cero.

La cuantización de 8 bits y las pasadas enteras limitan la resolución del relieve. El PNG mm16 representa con mayor precisión la escala objetivo exportada, pero esta versión mantiene el raster de ejecución de 8 bits y el número entero de pasadas.

Gris con N=8	Pasadas
255 (blanco)	0
128	4
0 (negro)	8



Mapa sintético para explicar niveles: no procede de medición ni inferencia.

5. IA y selección de dispositivo

comfy_design_client.workflow construye nodos integrados: CheckpointLoaderSimple, CLIPTextEncode, EmptyLatentImage o VAEEncode, KSampler, VAEDecode y SaveImage. generate envía /prompt, consulta /history y /queue y recupera /view.

comfy_setup.ensure_depth prioriza comfy_venv si compatible_gpu demuestra cálculo CUDA. Instala imports de profundidad que falten; ensure_depth_model valida y conserva el modelo común. Si esta ruta falla, ensure_depth_cpu utiliza depth_venv.

depth_estimate_cli.predict_depth mueve modelo y entradas a CUDA, calcula sin gradientes y recupera el tensor en CPU. Una excepción provoca inferencia CPU. El proceso exterior puede repetir en el entorno independiente CPU si el runtime elegido no termina correctamente.

El supervisor de ComfyUI recibe PID propietario, puerto y token. El cierre de la aplicación crea la señal de parada; el supervisor vigila también la muerte del proceso padre. Cada programa debe cerrar sus propios procesos.

Los botones Cancelar solicitan interrupción y no deben terminar hilos Qt por fuerza. SetupWorker mantiene el proceso de instalación y lo detiene; las señales status/result/failed actualizan la interfaz desde el hilo correcto.

6. Mapa objetivo en milímetros

passes_from_mm(target_mm, measured_mm, calibration_passes) valida valores positivos y finitos. Calcula tasa mm/pasada y ceil(target/tasa), con máximo 999. El modelo es lineal y sus entradas proceden de una muestra medida por el usuario.

export_mm_map convierte un mapa de 8 bits al PNG uint16: entero = round((255-gris)/255*65535). El JSON declara target_max_mm, fórmula, unidad, tipo desired_depth_not_measurement y calibración opcional.

ImageDepthDialog guarda depth_mm_enabled, depth_target_mm, depth_measured_mm, depth_calibration_passes, depth_mm_per_pass, depth_mm_map y depth_calibration_pen. Antes del marcado se recalcula el máximo y se comparan velocidad, potencia, frecuencia e intervalo con la pluma guardada.

La comparación no comprueba foco, aleación, espesor ni estado real de la pieza. Documenta esos datos en tu receta; no presentes el valor objetivo como medida conseguida. Una calibración de punto único no modela curvas no lineales.

Ejemplo: crear y leer una escala mm

```
from depth_map_tools import passes_from_mm, export_mm_map
from PIL import Image
import numpy as np, json
from pathlib import Path

# Solo números didácticos: no son una receta comprobada.
count, rate = passes_from_mm(0.20, 0.10, 10)
assert count == 20
output = export_mm_map('mapa.png', 0.20, 0.10, 10)
meta = json.loads(Path(output).with_suffix('.json').read_text())
with Image.open(output) as image:
    depth_mm = np.asarray(image, dtype=float) / 65535
    depth_mm *= meta['target_max_mm']
print(depth_mm.min(), depth_mm.max())
```

Para reproducirlo ejecuta demo_mapa_mm.py desde el entorno del editor. Ese ejemplo crea su propio mapa didáctico. La fórmula describe profundidad deseada; no mide una pieza.

7. Añadir una herramienta al editor

Los instaladores install_*(main) añaden acciones, barras y diálogos. Usa un módulo separado para mantener Main legible. Una acción que modifica geometría debe señalar changed y actualizar la vista previa.

```
from PySide6.QtGui import QAction
from PySide6.QtWidgets import QMessageBox

def install_counter(main):
    action = QAction('Contar objetos', main)
    def show_count():
        count = len(main.selected())
        QMessageBox.information(main, 'Selección',
                                f'{count} objetos seleccionados')
    action.triggered.connect(show_count)
    tools = next((a.menu() for a in main.menuBar().actions()
                  if a.menu() and
                  a.text().replace('&', '') == 'Tools'), None)
    if tools:
        tools.addAction(action)

# Llamar install_counter(self) desde Main.__init__
# después de crear los menús.
```

Para operaciones lentas usa señales y un worker; no introduzcas descargas o inferencia en el slot del botón. No accedas directamente a widgets desde un hilo secundario. La herramienta incluida añade un contador, sin tocar el láser.

8. Puente nativo y API C

El contrato está en BSL_SDK/43AYE3_G2_SDK/43AYE3_BSL_COMPAT.h. Las funciones se exportan con extern C, __cdecl y convención de retorno 0 = correcto / negativo = error. El texto se pasa como wchar_t con tamaño de buffer explícito.

AYE_GetVersion, AYE_GetLastError y AYE_GetCapabilities informan del wrapper. AYE_DeviceConnect y AYE_DeviceGetInfo gestionan sesión. AYE_Stop solicita parada. Las capacidades describen rutas implementadas, no que la máquina esté conectada.

AYE_RunRedBoxContinuous y AYE_RunRedVectorContinuous reciben HANDLE de evento STOP y proceso padre. AYE_MarkVectorDesign ejecuta el documento de marcado preparado. Son llamadas físicas: su uso debe respetar el ciclo de conexión y parada.

BSL_NewFile / BSL_AddLinesToFile2 / BSL_SetPenParam / BSL_SaveEntLibToFile preparan y guardan entidades nativas. BSL_SetPenParam utiliza frecuencia en Hz, mientras la interfaz introduce kHz. pointsXYFloat y pointCounts deben corresponder exactamente con lineCount.

No copies símbolos internos del fabricante como API estable. Las DLL y estructuras están ligadas al runtime incluido. Un cambio de arquitectura o versión requiere inspección de exports y pruebas de documento antes de validar la máquina.

Familia	Funciones principales
Información	AYE_GetVersion, AYE_GetLastError, AYE_GetCapabilities
Sesión	AYE_DeviceConnect, AYE_DeviceGetInfo, AYE_DeviceClose
Parada / rojo	AYE_Stop, AYE_RunRedBoxContinuous, AYE_RunRedVectorContinuous
Marcado	AYE_MarkLines, AYE_MarkVectorDesign
Documento	BSL_NewFile, BSL_AddLinesToFile2, BSL_AddBmpToFile, BSL_SaveEntLibToFile
Plumas	BSL_GetPenParam, BSL_SetPenParam

Ejemplo C++ de consulta de versión

```
#include <iostream>
#include "43AYE3_BSL_COMPAT.h"

int wmain() {
    wchar_t version[512] = {};
    const int result = AYE_GetVersion(version, 512);
    if (result != 0) return 1;
    std::wcout << version << L"\n";
    return 0;
}
```

Compilar y enlazar como Win32/x86 contra la biblioteca del wrapper. Este ejemplo consulta información; no conecta ni marca. No se ha compilado aquí porque el entorno de revisión es Linux. Los BAT de compilación del proyecto fijan el toolchain Windows y las rutas.

9. Compilación y diagnóstico

ARRANCAR_43AYE3_COMPLETO.bat llama a PREPARAR_IA_COMFYUI.ps1 y después comprueba SDK y ejecutables del puente. COMPILAR_SDK_43AYE3.bat produce el wrapper y selftest. COMPILAR_DIAGNOSTICO_G2.bat produce clientes de diagnóstico.

No mezcles arquitectura del editor con la del wrapper: Python x64 convive con ejecutables auxiliares x86. La carpeta G2_RUNTIME debe permanecer junto al código; G2PRO_VENDOR_DIR apunta a ella.

Los clientes device_state_readonly_client y device_active_safe_client separan consultas de estado y operaciones activas. Las sondas de marcado o emisión roja no equivalen a consultas de información: producen efectos físicos.

simulation_audit compara trabajo, campo y configuración sin transmitir. Un PASS de preparación valida geometría o llamada, pero no demuestra emisión visible ni resultado en material. Conserva resultado, código de salida y registro.

El arranque comprueba --queue-version y recompila el SDK si falta MARK_QUEUE_VERSION=1. Requiere Visual Studio Build Tools C++ x86/x64 en Windows. No se ha compilado el binario Windows en el entorno de revisión Linux; la lógica de lectura y sesión se ha compilado y probado con dobles de dispositivo. Al cambiar el código de geometría, comprueba que Line / Fill / Image y transformaciones conservan tamaño. Al cambiar capas, verifica ida y vuelta .43laser. Al cambiar IA, verifica cancelación, cierre y falta de modelo.

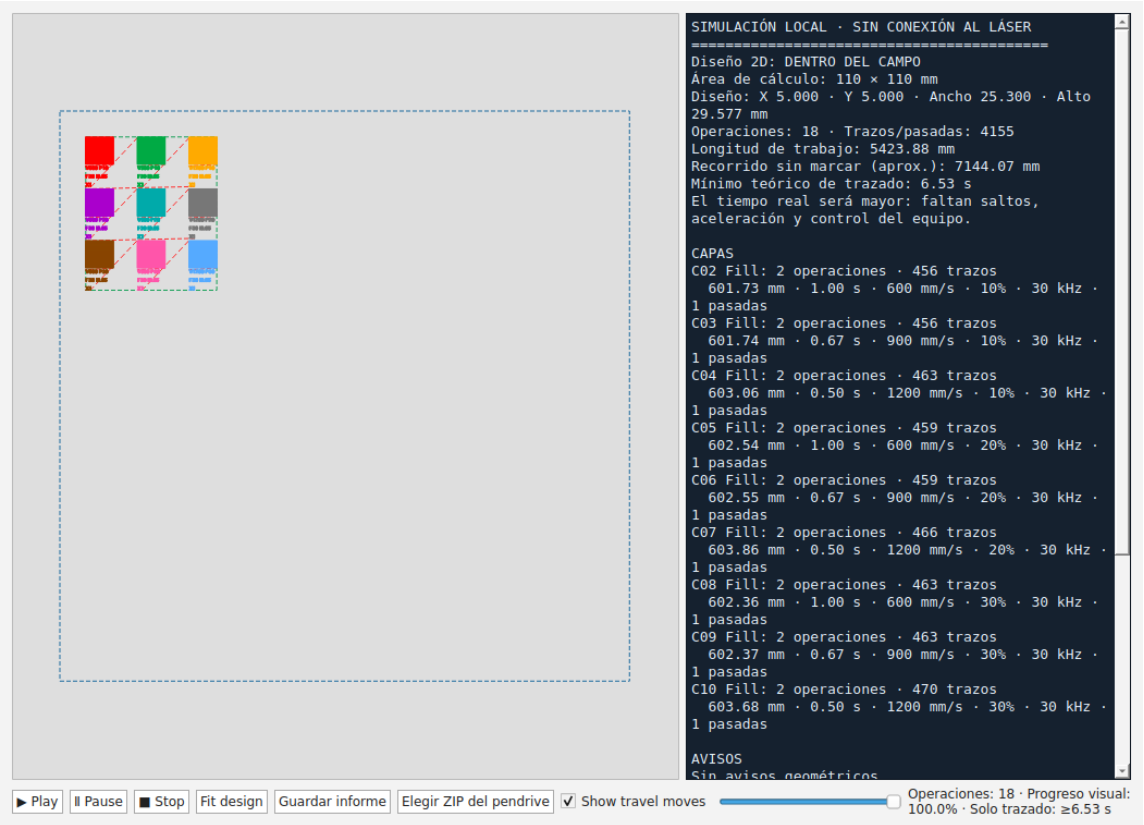
10. Ejemplos incluidos y pruebas

DOCUMENTACION/ejemplos/demo_trabajo.py crea un cuadrado Fill de 20 mm, construye LaserJob y guarda un .43laser sin conectar el dispositivo. demo_mapa_mm.py genera un mapa sintético con PNG de 16 bits y escala JSON. demo_herramienta.py contiene un instalador de QAction.

Para ejecutar ejemplos usa work/.venv/Scripts/python.exe. Los scripts añaden work a sys.path desde su ubicación. Conserva DOCUMENTACION dentro de la raíz del proyecto. No requieren volver a descargar el modelo IA para probar geometría o escala mm.

Validación realizada: sintaxis Python de los archivos modificados; ejecución de las pantallas en Qt offscreen; selección GPU / CPU y recuperación CUDA con dobles de prueba; cálculo mm y rechazo de entradas inválidas; lectura de extremos de PNG uint16; construcción de ejemplo y persistencia de atributos.

La revisión local no dispone de RTX 5060 ni máquina física. El registro aportado por el usuario confirma inferencia CUDA con su RTX 5060 Laptop; no valida el resultado físico del tallado. Las capturas muestran ventanas reales con datos de ejemplo. Las recetas y calibración deben probarse en tu instalación y material.



Vista previa real de la matriz y su informe de simulación.

11. Referencia del código completo

Inventario automático del código de aplicación del ZIP, después de esta actualización. Se incluyen los módulos de interfaz, imagen, geometría, IA, dispositivo y exportación. Las firmas son referencia de implementación: las funciones internas no son una API pública estable. La biblioteca estándar del Python portátil se excluye del inventario de funciones de la aplicación.

adjust_image_dialog.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class AdjustImageDialog

settings(self)

interval_changed(self, *_)

dpi_changed(self, *_)

schedule_render(self, *_)

_render_done(self, token, out, error)

render(self, *_)

apply_to_canvas(self)

reset(self)

bsl_orzx_export.py

Compatibilidad histórica: la exportación ORZX es ahora 100% nativa G2. No busca, copia ni ejecuta GLaser/BSLAppSimple.

class OrzxExportError

export_job(main, filename, sdk_dir=None, selected_only=False)

bsl_sdk_loader.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class DllCandidate

ready(self)

_u16(data: bytes, off: int)

_u32(data: bytes, off: int)

_cstr(data: bytes, off: int)

_parse_exports(path: Path)

_declared_43aye3_exports()

inspect_dll(path: Path)

scan_sdk(root: Path | str=SDKDIR)

detect_sdk_dll(root: Path | str=SDKDIR, require_x86: bool=True)

report(root: Path | str=SDKDIR)

write_manifest(path: Path, candidate: DllCandidate)

bsl_sdk_orzx.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class NativeOrzxError

_prepare_sdk_runtime()

_polylines(geometry)

_image_record(op)

_job_data(main, selected_only=False)

_write_u16_string(f, text)

_write_input(path, pens, lines_by_pen, images)

_build(timeout=180)

_ensure_bridge()

sdk_status()

export_job(main, filename, selected_only=False)

classic_ui.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

install_classic_layout(w)

comfy_design_client.py

Local ComfyUI API; no dependency on custom nodes or cloud services.

reload_endpoint()

api(route, data=None, timeout=30)

available_models()

upload_image(path)

workflow(prompt, style, model, width, height, steps=24, reference=None, strength=0.65, seed=None, circular=False)

generate(graph, notify=lambda message: None, cancelled=lambda: False)

prepare_artwork(data, path, style, width_mm, height_mm, circular=False, border=True)

comfy_design_ui.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class DesignWorker

run(self)

stop_owned_server()

class SetupWorker

stop(self)

run(self)

class DesignDialog

sync_circle(self, checked)

set_models(self, models)

connect_comfy(self)

setup_finished(self)

setup_message(self, message)

pick_reference(self)

clear_reference(self)

start_generation(self)

generation_finished(self)

show_result(self, path)

error(self, message)

cancel_generation(self)

import_design(self)

shutdown(self)

reject(self)

open_design(main)

install_ai_design(main)

comfy_server_supervisor.py

An owned ComfyUI server stops on request or when Laser Studio exits.

supervise(owner, port, token)

comfy_setup.py

Local, idempotent ComfyUI installation in this release's work folder.

runtime_env()

say(message)

run(command)

download(url, target)

models_at(url)

ensure_venv(folder)

ensure_app()

compatible_gpu(python)

nvidia_present()

ensure_cuda_runtime()

valid_model(path, minimum_bytes=1024 ** 3)

ensure_model(root)

ensure_comfy(owner=None, prepare_only=False)

ensure_depth()

ensure_depth_cpu()

ensure_depth_model(python)

main()

deep_engraving_dialog.py

Graphical setup for vector depth engraving on the G2.

class HatchPreview

paintEvent(self, event)

class DeepEngravingDialog

load_reference(self)

update_preview(self, *args)

apply_to(self, cut)

class ImageDepthDialog

accept(self)

apply_to(self, item)

depth_estimate_cli.py

Local Depth Anything V2 Small inference; automatic CUDA inference with CPU fallback.

report(message)

estimate(source, output)

predict_depth(model, inputs, torch)

depth_map_tools.py

Image-depth postprocessing and editable pass proposals; no physical Z claim.

depth_background_mask(source_image, remove_white=True)

normalize_depth_prediction(prediction, source_image)

passes_from_mm(target_mm, measured_mm, calibration_passes)

export_mm_map(map_path, target_mm, measured_mm=0, calibration_passes=1, output_dir=None)

prepare_depth_map(raw_path, source_path, out_path, invert=False, gamma=1.0, remove_white=True, circular=False, detail_weight=0.0)

propose_passes(cut, map_path=None)

depth_map_ui.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class DepthWorker

start(self)

read_output(self)

finish_once(self)

complete(self, code, *args)

error(self, error)

isRunning(self)

stop(self)

wait(self, milliseconds)

class DepthMapDialog

set_source(self, path, snapshot=None)

use_selected(self)

open_file(self)

circle_changed(self, checked)

message(self, message)

generate(self)

worker_finished(self)

ready(self, path)

refresh_map(self, *args)

send_to_depth(self)

cancel(self)

shutdown(self)

reject(self)

open_depth_map(main)

install_depth_map(main)

device_active_safe_client.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

_build(base, timeout=120)

run_device_active_safe_probe(base=None, vendor_dir=None, timeout=35)

device_state_readonly_client.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

_build(base, timeout=120)

_values(lines)

_int(v, default=-999999)

run_device_state_readonly_probe(base=None, vendor_dir=None, timeout=25)

documentation_help.py

Ayuda local: abre manuales empaquetados sin conexión a Internet.

documentation_dir()

open_document(parent, filename=None)

install_help(main, menu)

functional_controls.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

install_functional_controls(w)

g2_native_frame.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class Calibration

load_calibration()

save_calibration(c: Calibration)

_scene_items(main)

frame_bounds(main)

mark_probe_center(main)

_map_logical(x_mm, y_mm, c: Calibration)

make_rect_path(rect, c: Calibration)

write_path(main)

_build(base=APP, timeout=120)

start_frame(main, vendor_dir)

g2_runtime.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

discover()

diagnostic()

g2_usb_diagnostic.py

Comprobacion Windows PnP solamente. No abre el dispositivo ni carga DLL vendor.

classify(entries)

diagnose(runner=subprocess.run, platform=os.name)

main()

g2_vector_frame.py

Convert one selected QPainterPath into a bounded G2 red outline.

selected_vector_points(main)

all_vector_paths(main, layer_filter=None)

layer_mark_paths(main, layer)

backend_batches(paths)

raster_mark_paths(item)

depth_map_paths(item, passes, interval)

geometry_pro.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

path_from_points(points, closed=False)

regular_polygon(cx, cy, r, n, rotation=-90)

star(cx, cy, r1, r2, n, rotation=-90)

union(paths)

intersect(paths)

subtract(base, cutters)

xor(paths)

hatch_segments(path, interval=0.1, angle_deg=0, cross=False)

glaser_runtime.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

ensure_runtime(*args, **kwargs)

hatch_engine.py

Rayado de contornos SVG con aristas activas por línea de barrido.

horizontal_hatch(path, interval=0.1, angle=0.0)

image_async.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class _Signals

class PrepareTask

run(self)

submit_prepare(token, source, kwargs, callback)

image_controller.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

install_image_controller(w)

image_display_utils.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

load_display_pixmap(path, max_side=DISPLAY_MAX_SIDE)

_move_scene_delta(item, dx, dy)

replace_pixmap_keep_scene_bounds(item, pixmap)

image_export_native.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

_safe_name(text: str)

_is_helper_item(item)

_selected_or_all(scene, area: str)

_bounds(items)

class _SceneExportState

export_scene(window, fmt='PNG', area='SELECCION', color_mode='ORIGINAL', bg_mode='BLANCO', dpi=600, margin_px=0)

class ImageExportDialog

_do_export(self)

image_worker_cli.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

main()

interactive_items.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class ResizeHandle

mousePressEvent(self, e)

mouseMoveEvent(self, e)

mouseReleaseEvent(self, e)

class InteractiveMixin

setup_interactive(self)

update_handles(self)

itemChange(self, change, value)

contextMenuEvent(self, e)

resize_from_handle(self, corner, p)

marking_preview_file(source, darkness=0, contrast=1.15, gamma=0.95, sharpen=1.2)

laser_image_pipeline.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

_kernel_arrays(kernel)

_diffuse_python(a, kernel)

diffuse(a, kernel, mode_name=None)

_clustered_halftone(a, cell)

process_array(source, mode='Grayscale', brightness=0, darkness=0, contrast=0, gamma=1.0, sharpen=0, invert=False, threshold=128)

prepare(source, mode='Grayscale', brightness=0, darkness=0, contrast=0, gamma=1.0, sharpen=0, invert=False, threshold=128, dpi=254)

interval_from_dpi(dpi)

acceleration_name()

laser_job.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

_vector_image(path)

_raster_geometry(item, cut)

class JobOperation

class LaserJob

from_scene(main, selected_only=False, use_cache=False)

laser_studio.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class Cut

dict(self)

class GItem

setPath(self, path)

shape(self)

mouseDoubleClickEvent(self, event)

data43(self)

class RasterItem

data43(self)

class GroupItem

members(self)

boundingRect(self)

shape(self)

data43(self)

class Scene

drawBackground(self, p, r)

mousePressEvent(self, e)

mouseMoveEvent(self, e)

mouseReleaseEvent(self, e)

class View

wheelEvent(self, e)

mouseMoveEvent(self, e)

mousePressEvent(self, e)

mouseReleaseEvent(self, e)

class CutDialog

accept(self)

class Main

act(self, text, slot, shortcut=None, check=False)

_dedupe_shortcuts(self)

make_actions(self)

make_menus(self)

make_tools(self)

make_layers(self)

make_laser(self)

check_g2_usb(self)

usb_check_finished(self, code, status)

usb_check_error(self, error)

make_palette(self)

tool(self, t)

keyPressEvent(self, e)

normalize_layout(self)

selected(self)

select_all(self)

closeEvent(self, e)

apply_layer(self, it)

fill_selected(self)

set_layer(self, i, assign_selection=True)

edit_cut(self, i)

delete(self)

duplicate(self)

rotate(self, a)

flip(self, h)

align(self, k)

distribute(self, h)

group(self)

ungroup(self)

grid_array(self)

circular_array(self)

offset(self)

boolean_union(self)

measure(self)

material_test(self)

material_library(self)

add_material_parameters(self)

vectorize_image(self)

adjust_image(self)

preview(self)

text_editor(self, item=None)

schedule_preview(self)

begin_transform(self)

end_transform(self)

refresh_preview(self)

no_device(self)

fit(self)

refresh_status(self)

ensure_cut_capacity(self, count)

update_cut_row(self, layer)

reset_layout(self)

_confirm_replace_project(self)

new(self)

save(self)

saveas(self)

_save(self, fn)

open(self)

import_any(self)

import_svg_file(self, fn)

import_svg(self)

export_orzx(self)

export_svg(self)

lightburn_workflow.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

_act(tb, text, tip, fn, shortcut=None)

install_lightburn_workflow(w)

mark_queue.py

Prepare a native mark queue preserving layer parameters and block order.

write_mark_queue(groups, directory)

material_library.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

_load()

_table(cols, rows)

_append(t, row=None)

_rows(t)

class ManualPresetDialog

values(self)

accept(self)

class MaterialLibraryDialog

refresh(self)

add_parameters(self)

selected_values(self)

add_preset(self)

edit_hint(self)

_sync_visible_edits(self)

duplicate(self)

delete(self)

manage_liquids(self)

save(self)

_write_library(self)

export_json(self)

import_json(self)

apply_preset(self)

_replace_table(old, cols, rows)

open_library(main, add_parameters=False)

material_test_matrix.py

Matriz de prueba: una capa Fill y una etiqueta vectorial por cuadrado.

label_for_cut(cut)

_label_path(label, width)

refresh_labels(main, layer)

generate_matrix(main, rows, cols, speed_range, power_range, frequency_range, interval, passes, material='', liquid='')

group_matrix(main)

native_probe.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

pe_machine(path)

printable_strings(path, minlen=5)

inspect(base='.')

windows_pnp()

production_laser_panel.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

validate_mark_layer_mode(mode, has_vectors, has_images)

_laser_menu(window)

_hide_test_panels(host)

class CalibrationDialog

defaults(self)

accept(self)

install_production_laser_panel(w)

professional_ui.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

install_professional_ui(w)

update_numeric(w)

apply_numeric(w)

make_shape_properties(w)

make_move_dock(w)

make_library_dock(w)

make_variable_text_dock(w)

make_optimization_action(w)

optimization_dialog(w)

make_devices_action(w)

devices_dialog(w)

install_v4_tools(w)

install_v5_image_vector(w)

install_v6_scale_materials(w)

raster_tools.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

load_rgba(path)

adjust(img, brightness=1.0, contrast=1.0, gamma=1.0, sharpen=0.0, invert=False)

grayscale_array(img)

trace_contours(img, threshold=128, min_area=8, epsilon=0.002, alpha=False)

brightness_from_slider(value)

darkness_from_slider(value)

rulers_live.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class Ruler

setUnit(self, u)

paintEvent(self, e)

process_preview(source, mode='Grayscale', brightness=0, contrast=0, gamma=1.0, sharpen=0, invert=False, threshold=128)

install_rulers_live_image(w)

simulation_audit.py

Auditoría local del trabajo; nunca abre el puente ni envía órdenes al G2.

_factory_profile(configured_path, factory_zip=None)

_configuration(factory_zip=None)

_operation_metrics(op)

audit(main, job, factory_zip=None)

svg_importer.py

Importa contornos SVG preservando subtrazos, curvas y dimensiones fisicas.

_length_mm(value)

_viewport(path)

load_svg_paths(filename)

text_editor.py

Editor de texto no modal. Los cambios se reflejan en el lienzo y Preview.

open_text_editor(main, item=None)

text_tools.py

Contornos de texto editables con dimensiones fisicas en milimetros.

text_path(text, font_family='Arial', size_mm=8, bold=False, italic=False, tracking_mm=0, line_spacing=1.2, align='Left', mirror_h=False, mirror_v=False)

vectorize_image.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class VectorizeDialog

_load_gray(path)

_binary(img, threshold, blur, invert)

_contour_path(binary, simplify, min_area)

_centerline_path(binary, simplify, min_area)

vectorize_selected(app)

visual_preview.py

Módulo de implementación del proyecto; revisar las llamadas desde Main y el panel correspondiente.

class PreviewDialog

choose_factory_zip(self)

_add_path(self, path, layer, kind, speed, passes)

build(self)

fit_content(self)

resizeEvent(self, event)

render_progress(self, v)

tick(self)

reset(self)

```
save_audit(self)
```

12. Contrato nativo completo

```
AYE_API int __cdecl AYE_GetVersion(wchar_t* outText, int cch);

AYE_API int __cdecl AYE_GetLastError(wchar_t* outText, int cch);

AYE_API int __cdecl AYE_GetCapabilities(unsigned int* outFlags);

AYE_API int __cdecl AYE_DeviceConnect(void);

AYE_API int __cdecl AYE_DeviceGetInfo(AYE_DeviceInfo* info);

AYE_API int __cdecl AYE_Stop(void);

AYE_API int __cdecl AYE_ProbeMarker(void);

AYE_API int __cdecl AYE_ProbeRedPaths(int mode, int* pathCount);

AYE_API int __cdecl AYE_TestRedViaDoMark(void);

AYE_API int __cdecl AYE_TestRedOutlineViaDoMark(void);

AYE_API int __cdecl AYE_RunRedBoxContinuous(HANDLE stopEvent, HANDLE parent);

AYE_API int __cdecl AYE_RunRedVectorContinuous(HANDLE stopEvent, HANDLE parent);

AYE_API int __cdecl AYE_FramePolyline(const AYE_PointD* pts, int count, int repeats,
     int delayMs);

AYE_API int __cdecl AYE_DeviceClose(void);

AYE_API int __cdecl AYE_MarkLines(const AYE_PointD* pts, int count, int pen);

AYE_API int __cdecl AYE_MarkVectorDesign(HANDLE parent);

AYE_API int __cdecl BSL_NewFile(wchar_t* filePath);

AYE_API int __cdecl BSL_SaveEntLibToFile(wchar_t* filePath);

AYE_API int __cdecl BSL_UnLoadDataFile(wchar_t* filePath);

AYE_API int __cdecl BSL_AddLinesToFile2(wchar_t* filePath, void* pointsXYFloat, int
    lineCount, int* pointCounts, wchar_t* entityName, int pen);

AYE_API int __cdecl BSL_AddBmpToFile(wchar_t* filePath, wchar_t* imagePath, wchar_t*
     entityName, double widthMm, double heightMm, double centerX, double centerY,
    double rotationDeg, int pen);

AYE_API int __cdecl BSL_GetPenParam(wchar_t* filePath, unsigned int pen, int* loops,
     double* speed, double* power, double* current, int* frequencyHz, int* qPulse,
    int* startTc, int* laserOffTc, int* endTc, int* polygonTc, double* jumpSpeed,
    int* jumpPosTc, int* jumpDistTc, double* endComp, bool* pulsePoint, int*
    pulseCount, float* pulseTime);

AYE_API int __cdecl BSL_SetPenParam(wchar_t* filePath, unsigned int pen, int loops,
    double speed, double power, double current, int frequencyHz, int qPulse, int
    startTc, int laserOffTc, int endTc, int polygonTc, double jumpSpeed, int
    jumpPosTc, int jumpDistTc, double endComp, bool pulsePoint, int pulseCount,
    float pulseTime);
```

13. Inventario de archivos del proyecto

Este inventario incluye los archivos del editor, scripts de arranque, configuración, fuentes nativas y herramientas. AI_RUNTIME/python312 se documenta como dependencia empaquetada; sus miles de archivos no se reproducen individualmente. G2_RUNTIME se incluye como runtime binario del fabricante y requiere mantener la carpeta íntegra.

BSL_SDK

BSL_SDK/43AYE3_G2_SDK/43AYE3_BSL_COMPAT.cpp

BSL_SDK/43AYE3_G2_SDK/43AYE3_BSL_COMPAT.def

BSL_SDK/43AYE3_G2_SDK/43AYE3_BSL_COMPAT.h

BSL_SDK/43AYE3_G2_SDK/sdk_selftest_x86.cpp

BSL_SDK/43AYE3_G2_SDK/tools/analyze_g2_frame_transaction.py

BSL_SDK/43AYE3_G2_SDK/tools/analyze_g2_mark_capture.py

BSL_SDK/43AYE3_G2_SDK/tools/analyze_g2_red_pcap.py

BSL_SDK/43AYE3_G2_SDK/tools/compare_g2_red_shapes.py

BSL_SDK/43AYE3_G2_SDK/tools/g2_red_packet_codec.py

BSL_SDK/43AYE3_G2_SDK/tools/verify_g2_frame_capture.py

BSL_SDK/43AYE3_G2_SDK/tools/verify_red_emitter.py

BSL_SDK/43AYE3_G2_SDK/tools/verify_red_geometry.py

BSL_SDK/43AYE3_G2_SDK/tools/verify_seamark_red_mode.py

G2_RUNTIME

G2_RUNTIME/43aye3_runtime.json

G2_RUNTIME/CrashRpt1403.dll

G2_RUNTIME/DrawUI.dll

G2_RUNTIME/Font/#U5bd2#U9e70#U4e2d#U6587.shx

G2_RUNTIME/Font/#U5de5#U7a0b#U6c49#U5b57.shx

G2_RUNTIME/Font/#U659c#U4f53#U897f#U6587.shx

G2_RUNTIME/Font/ARCHD.SHX

G2_RUNTIME/Font/ARCHF.SHX

G2_RUNTIME/Font/ARCHS.SHX

G2_RUNTIME/Font/CADSLDXJ.SHX

G2_RUNTIME/Font/CJB.shx

G2_RUNTIME/Font/China_ds.shx

G2_RUNTIME/Font/HMDXST.SHX

G2_RUNTIME/Font/Romanc.shx

G2_RUNTIME/Font/Romand.shx

G2_RUNTIME/Font/ST64S.shx

G2_RUNTIME/Font/Simtxt.shx

G2_RUNTIME/Font/Sinotxt1.shx

G2_RUNTIME/Font/Stdb1.shx

G2_RUNTIME/Font/Stkx.shx

G2_RUNTIME/Font/XC01.SHX

G2_RUNTIME/Font/XIAO.SHX

G2_RUNTIME/Font/hzdx.shx

G2_RUNTIME/Font/hztxt.shx

G2_RUNTIME/Font/hztxt0.shx

G2_RUNTIME/Font/romans.shx

G2_RUNTIME/Font/scripts.shx

G2_RUNTIME/Font/simplex.shx

G2_RUNTIME/Font/txt.shx

G2_RUNTIME/LPCodeAddin.dll

G2_RUNTIME/Lang/lang_chs.ini

G2_RUNTIME/Lang/lang_cht.ini

G2_RUNTIME/Lang/lang_cz.ini

G2_RUNTIME/Lang/lang_en.ini

G2_RUNTIME/Lang/lang_esp.ini

G2_RUNTIME/Lang/lang_far.ini

G2_RUNTIME/Lang/lang_fra.ini

G2_RUNTIME/Lang/lang_ger.ini

G2_RUNTIME/Lang/lang_jpn.ini

G2_RUNTIME/Lang/lang_kor.ini

G2_RUNTIME/Lang/lang_por.ini

G2_RUNTIME/Lang/lang_rus.ini

G2_RUNTIME/Lang/lang_tha.ini

G2_RUNTIME/Lang/lang_tur.ini

G2_RUNTIME/Lang/lang_uae.ini

G2_RUNTIME/Lang/lang_vie.ini

G2_RUNTIME/MarkUI.dll

G2_RUNTIME/ParaUI.dll

G2_RUNTIME/Plug/markcfg0

G2_RUNTIME/SeaBase.dll

G2_RUNTIME/SeaCali.dll

G2_RUNTIME/SeaDraw.dll

G2_RUNTIME/SeaExts.dll

G2_RUNTIME/SeaMark.dll

G2_RUNTIME/SeaUI.dll

G2_RUNTIME/calib.dll

G2_RUNTIME/config/AutoSave/BslCAD.cfg

G2_RUNTIME/config/AutoSave/DlgPar.cfg

G2_RUNTIME/config/AutoSave/LmcPar.cfg

G2_RUNTIME/config/AutoSave/fillPara.cfg

G2_RUNTIME/config/BslCAD.cfg

G2_RUNTIME/config/DlgPar.cfg

G2_RUNTIME/config/LmcPar.cfg

G2_RUNTIME/config/fillPara.cfg

G2_RUNTIME/crashrpt_lang.ini

G2_RUNTIME/crypto3.dll

G2_RUNTIME/crypto5.0.dll

G2_RUNTIME/crypto6.0.dll

G2_RUNTIME/dbghelp.dll

G2_RUNTIME/libcodebar.dll

G2_RUNTIME/mfc100u.dll

G2_RUNTIME/msvcp100.dll

G2_RUNTIME/msvcr100.dll

G2_RUNTIME/res/login.bmp

G2_RUNTIME/res/logo.ico

G2_RUNTIME/ucrtbase.dll

Raíz

ARRANCAR_43AYE3_COMPLETO.bat

ARRANCAR_43AYE3_SEGURO.bat

COMPILAR_DIAGNOSTICO_G2.bat

COMPILAR_SDK_43AYE3.bat

COMPILAR_SDK_BSL.bat

INSTALAR_ACELERACION_IMAGEN.bat

KNOWN_ISSUES.md

LEEME_IA_COMFYUI.txt

LEEME_V1.txt

PREPARAR_IA_COMFYUI.ps1

adjust_image_dialog.py

bsl_orzx_export.py

bsl_sdk_loader.py

bsl_sdk_orzx.py

classic_ui.py

comfy_design_client.py

comfy_design_ui.py

comfy_server_supervisor.py

comfy_setup.py

deep_engraving_dialog.py

depth_estimate_cli.py

depth_map_tools.py

depth_map_ui.py

device_active_safe_client.py

device_state_readonly_client.py

documentation_help.py

functional_controls.py

g2_frame_calibration.json

g2_native_frame.py

g2_native_orzx.py

g2_runtime.py

g2_usb_diagnostic.py

g2_vector_frame.py

geometry_pro.py

glaser_runtime.py

hatch_engine.py

image_async.py

image_controller.py

image_display_utils.py

image_export_native.py

image_worker_cli.py

interactive_items.py

laser_image_pipeline.py

laser_job.py

laser_studio.py

lightburn_workflow.py

mark_queue.py

material_library.py

material_presets.json

material_test_matrix.py

native_probe.py

production_laser_panel.py

professional_ui.py

raster_tools.py

reparar_gpu.py

rulers_live.py

simulation_audit.py

svg_importer.py

text_editor.py

text_tools.py

vectorize_image.py

visual_preview.py

work_guardia_civil_25mm.svg

native_bridge_x86

native_bridge_x86/device_active_safe_x86.cpp

native_bridge_x86/device_bsl_sdk_bridge_x86.cpp

native_bridge_x86/device_config_probe_x86.cpp

native_bridge_x86/device_frame_native_x86.cpp

native_bridge_x86/device_state_readonly_x86.cpp

14. Implementación actual: proceso IA, mezcla y Help

DepthWorker es un QObject que supervisa QProcess; no es un QThread de inferencia. QProcess inicia el Python preparado, recibe salida y errores, y permite cancelar el proceso. Si el modelo y entorno GPU ya están disponibles se usa --run directamente para evitar preparación repetida.

El slot del diálogo se llama worker_finished. No lo llames finished: QDialog ya tiene una señal con ese nombre. Las conexiones y el arranque están protegidos y los botones recuperan su estado en caso de error.

normalize_depth_prediction normaliza percentiles 1 y 99 dentro de la figura, excluyendo el fondo exterior. depth_background_mask usa Pillow floodfill sobre un borde acolchado; no requiere OpenCV en el entorno CUDA.

prepare_depth_map combina IA y gris original con peso w: mezcla = (1-w)*IA + w*gris. Después aplica inversión, potencia gamma, exclusión del fondo y círculo opcional. Los ajustes depth_map_method y depth_original_gray_weight se guardan con la imagen.

documentation_help.documentation_dir resuelve DOCUMENTACION junto a work o dentro de work. open_document abre una URL local con QDesktopServices y muestra una ruta recuperable si falta el archivo o falla la asociación. install_help añade F1, tres PDF y dos accesos a carpetas.

Ejemplo reproducible: mezcla artística sin inferencia

```
from depth_map_tools import prepare_depth_map

# Archivos de ejemplo incluidos en DOCUMENTACION/ejemplos.
prepare_depth_map('demo_grises.png', 'demo_grises.png',
                  'mapa_artistico.png', gamma=1.0,
                  remove_white=True, circular=True,
                  detail_weight=0.60)
# 60 % de luminosidad original. No genera una medición 3D.
```

La mezcla puede ejecutarse con el entorno del editor; no necesita descargar modelos. En datos reales usa mapa IA como primer archivo y la imagen original como segundo.